

أوجد الترتيب

- الحد الزمني: 10 دقائق
- البيئة: وحدة GPU واحدة (16 GB VRAM)، من دون إنترنت
- حجم الحل: `solution.ipynb` ≤ 1 MB
- مساحة التخزين: 5 GB

المسألة

تُعطى حوارات منطوقة باللغة الإنجليزية بين مشاركين، *المتحدث A* و*المتحدث B*. يُقسّم كل حوار إلى مداخلات للمتحدثين، بحيث تحتوي كل مداخلة على كلام متحدث واحد فقط. تُخزّن كل مداخلة في ملف صوتي منفصل بصيغة `.wav`، ولذلك يُمثّل الحوار الكامل بمجموعة من ملفات `.wav`، ملف واحد لكل مداخلة.

لسوء الحظ، خلطت المداخلات عشوائيًا، لذلك لم تعد المحادثة منطقية. في اسم الملف `chunk_{k}.wav`، يشير `k` إلى المقطع ذي الترتيب `k` في المجموعة المخلوطة، وليس إلى المداخلة ذات الترتيب `k` في الحوار الأصلي.

!! مهمتك هي إعادة بناء الترتيب الزمني الأصلي للمحادثة.



مجموعة البيانات

يحتوي كل حوار على ملفات صوتية n مسماة `chunk_0.wav`، `chunk_1.wav`، ...، `chunk_{n-1}.wav`. المقاطع هي مداخلات منفردة. لا تتوافق أسماء الملفات إلا مع الترتيب المخلوط. وهي لا تشير إلى موضع المقطع في المحادثة الأصلية. يحتوي كل حوار على 7-20 مقطعًا، أحادي القناة، بتردد 44.1 kHz (يمكنك إعادة أخذ العينات).

يحتوي `prefix.json` على فهرس أسماء الملفات لأول مقطعين في كل حوار. يحدد هذا البداية الحقيقية للحوار ويزيل الالتباس بين قراءة المحادثة إلى الأمام أو إلى الخلف.

على سبيل المثال: يعني `[7, 12]` : 11 أن المداخلتين الأولى والثانية من الحوار 11 هما `chunk_7.wav` و `chunk_12.wav`، على الترتيب.

ما تحصل عليه

تتلقى مجلدين بالتنسيق نفسه:

المجلد	الحوارات	answers.json؟	استخدمه من أجل
dataset/train/	1,288	متضمن ☐	تدريب نموذجك / ضبطه الدقيق
dataset/test_public/	100	متضمن ☐	تشغيل مسار المعالجة وحساب نتيجتك محليًا

في أثناء التقييم، يُستبدل مجلد `dataset/test_public/` لديك بـ `hidden evaluation set` (`test_leaderboard_a` للوحة المتصدرين العامة و `test_leaderboard_b` للوحة المتصدرين النهائية) — ولهما الحجم والتنسيق نفسيهما مثل `dataset/test_public/` ولكن من دون `answers.json`.

يُنَفَّذ notebook الخاص بك مرة أخرى على تلك البيانات، ويُستخدم ملف `answers.json` الذي ينتجه لحساب النتيجة. تأتي حوارات الاختبار المحجوبة من التوزيع نفسه الذي يأتي منه `train`، ولذلك تمثل نتيجة `test_public` المحلية لديك معاينة موثوقة.

```

dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

```

المخرجات

لكل حوار، حدّد الترتيب الزمني الأصلي لمقاطع الصوتية. يجب أن يكون توقعك تبديلاً P لـ $\{0, 1, \dots, n-1\}$ ، حيث يكون $P[i]$ هو الموضع الزمني المتوقع لـ $chunk_i.wav$ (الأول = 0).

يجب أن يربط ملف المخرجات `answers.json` كل معرّف حوار بالتبديل المتوقع له:

```

{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}

```

مثال

يحتوي حوار على 3 مقاطع مخلوطة `chunk_0`, `chunk_1`, `chunk_2`:

المقطع المخلوط	المحتوى المنطوق	الموضع الحقيقي (الترتبة)
<code>chunk_0.wav</code>	"No worries — I'll send you the notes afterwards."	2 (الأخير)
<code>chunk_1.wav</code>	"Hey, are you coming to the three o'clock meeting?"	0 (الأول)
<code>chunk_2.wav</code>	"I can't — I've got a dentist appointment then."	1

الترتيب الحقيقي هو $\text{chunk}_0 \rightarrow \text{chunk}_1 \rightarrow \text{chunk}_2$ ، ولذلك $P = [2, 0, 1]$ ، ويحتوي `prefix.json` على [1, 2].

△ يجب أن يكون P تبديلاً حقيقياً: بطول n ، ومفهرساً بدءاً من 0، وتظهر فيه كل قيمة مرة واحدة بالضبط. تؤدي القيم المكررة أو المفقودة أو الواقعة خارج النطاق (مثل الفهرسة بدءاً من 1) إلى نتيجة 0 لذلك الحوار، وينطبق الأمر نفسه على أي حوار مفقود من الملف. يُرفض أي ملف غير صالح البنية أو ليس بتنسيق JSON.

حساب النتيجة

يُحسب تقييم هذه المهمة باستخدام دقة الترتيب الزوجي. يفحص كل زوج من المقاطع ويسأل: أي المقطعين يجب أن يأتي أولاً؟ يكون الزوج صحيحاً إذا أعطى توقعك الإجابة نفسها التي يعطيها الترتيب الحقيقي. بالنسبة إلى حوار يحتوي على n مقطعاً، يوجد

$$M = n(n - 1)/2$$

زوجاً؛ ولتكن I عدد الانعكاسات — أي الأزواج المرتبة بصورة مختلفة عن الترتيب الحقيقي:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i النتيجة النهائية هي متوسط نتائج الحوارات المنفردة عبر جميع الحوارات في التقسيم.

النماذج المسموح بها

لا يمكنك استخدام سوى النماذج المدربة مسبقاً التالية لحل هذه المهمة، سواء في أثناء التدريب أو التقييم. جميع هذه النماذج منزلة مسبقاً ومتاحة في البيئة. يمكنك الاطلاع على أمثلة لكيفية استخدامها في `baseline notebook` `solution.ipynb`. يُرجى ملاحظة أنه لا يمكنك استخدام أي نموذج آخر، وأن برنامجك لا يملك اتصالاً بالإنترنت.

- تمثيلات الكلام: **wav2vec 2.0**. يمكن أيضاً استخدام **Whisper encoder** بوصفه مستخرجاً للسمات. [بطاقة نموذج wav2vec](#)
- التعرف التلقائي على الكلام (ASR): **OpenAI Whisper** (بأي حجم). [بطاقة نموذج Whisper](#)
- النموذج اللغوي: **Qwen2.5-0.5B**، ويمكن استخدامه إما بطريقة zero-shot أو بعد ضبطه دقيقاً على تقسيم `train` المقدم. [بطاقة نموذج Qwen](#) لاحظ أن الحد الزمني البالغ 10 دقائق يجب أن يشمل أي تدريب أو ضبط دقيق تجريه في وقت التقييم، بالإضافة إلى الاستدلال على مجموعة التقييم.

كيفية التسليم

- افتح `solution.ipynb` وشغل جميع الخلايا. تأكد من أنه يكتب `answers.json` في مجلد العمل، مع تبديل لكل حوار في `dataset/test_public/` (100 حوار). في وقت التقييم، يُعاد تشغيل `notebook` على مجموعة الاختبار المخفية، ويُقِيم ملف `answers.json` الذي ينتجه هناك.
- حسن الحل إن أردت — أو لا تفعل؛ فالـ `baseline` وحده يتحقق من صحة مسار المعالجة.
- افتح تبويب `Git` في الشريط الجانبي الأيسر من `JupyterLab`.
- نفذ `Stage` لـ `solution.ipynb` (أيقونة + المجاورة له).
- أدخل رسالة `commit` وانقر على **Commit**.
- انقر على أيقونة السحابة ذات السهم المتجه إلى أعلى لإجراء `push`.

• عُد إلى صفحة المسابقة هذه وانقر على **.Submit**

سَلِّمْ ملفًا واحدًا بالضبط، باسم `solution.ipynb`.